

Project Team 5

Jason Miller

Jackson Benes

Logan Calloway

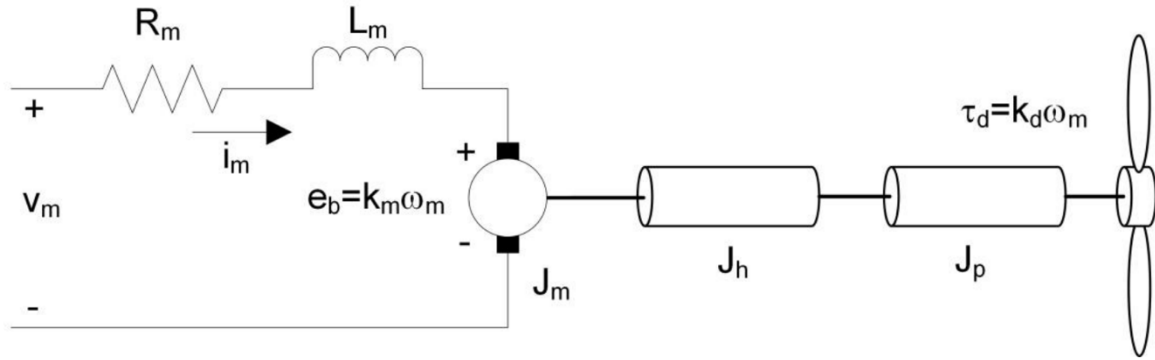
The Quanser Aero

The Quanser Aero (figure 1) is a dual motor machine that is intended to demonstrate mechatronics and control concepts at an undergraduate level. It contains two motors attached to propellers that exist on a rigid arm that is free to move in pitch and yaw. The propellers can be loosened and rotated to face any direction within 360 degrees. Figure 1 – A picture of the Quanser Aero.



The Thruster Model

The Quanser Aero has two thruster modules, each of which is driven by a DC motor. The motor armature circuit schematic for one of the thrusters is shown in Figure 2 and the electrical and mechanical parameters are given in Table 1. The DC motor shaft is connected to the propeller hub. The hub is a collet clamp used to mount the propeller to the motor and has a moment of inertia of J_h . A propeller is attached to the output shaft with a moment of inertia of J_p .



The back-emf voltage $e_b(t)$ depends on the speed of the motor shaft, $\omega_m(t)$, and the back-emf constant of the motor, k_m . It opposes the current flow. The back emf voltage is given by:

$$e_b(t) = k_m \omega_m(t)$$

The torque exerted by drag and air resistance has been simplified into an experimentally derived coefficient of the speed of the propeller. In this model the drag torque $\tau_d(t)$ which opposes the motor torque is given by:

$$\tau_d(t) = k_d \omega_m(t)$$

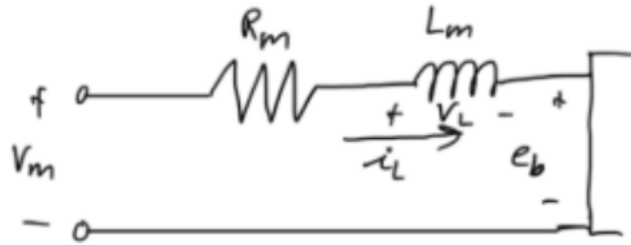
Note that the torque and motor back-emf constant is just the reciprocal of the coupling coefficient we discussed in class for an E-R transducer.

Symbol	Description	Value
DC Motor		
R_m	Armature resistance	$8.4 \, \Omega$
k_t	Torque constant	$0.042 \, \text{N} \cdot \text{m/A}$
k_m	Motor back-emf constant	$0.042 \, \text{V}/(\text{rad/s})$
J_m	Rotor inertia	$4.0 \times 10^{-6} \, \text{kg} \cdot \text{m}^2$
L_m	Armature inductance	$1.16 \, \text{mH}$
Propeller		
k_d	Drag/Air resistance	$1 \times 10^{-5} \, \text{N} \cdot \text{m}/(\text{rad/s})$
J_h	Propeller hub inertia	$3.04 \times 10^{-9} \, \text{kg} \cdot \text{m}^2$
J_p	Propeller inertia	$7.2 \times 10^{-6} \, \text{kg} \cdot \text{m}^2$

Assumptions

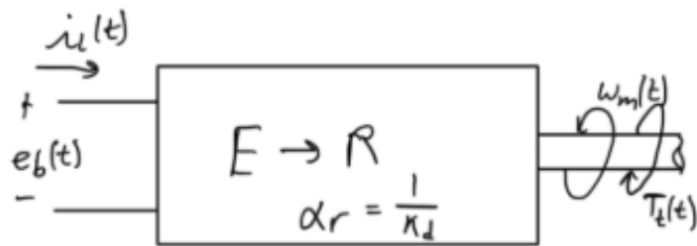
- Damping caused by friction is negligible.
- A positive voltage for $e_b(t)$ corresponds to a positive angular velocity for $\omega_m(t)$.

Derive the thruster model as a single-input single-output linear differential equation. Make sure the model has the angular velocity $\omega_m(t)$ as the output and the applied voltage $v_m(t)$ as the input. There should be no unknown parameters. Make sure you normalize your differential equation so that the highest order derivative has a coefficient of 1.



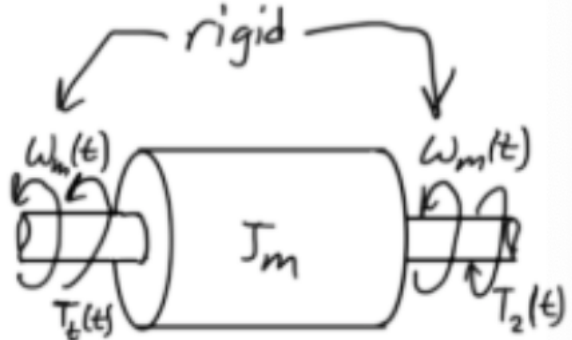
$$V_L(t) = L_m \cdot \frac{di_L(t)}{dt}$$

$$KVL: -V_m(t) + R_m i_L(t) + L_m \cdot \frac{di_L(t)}{dt} + e_b = 0$$



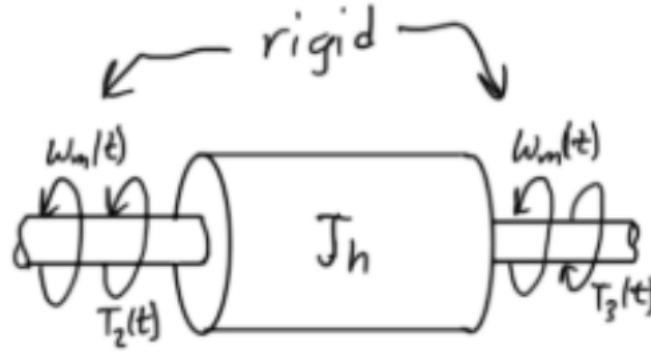
$$e_b(t) = k_m(t) \omega_m(t)$$

$$i_L(t) = \frac{1}{k_t} T_t(t)$$



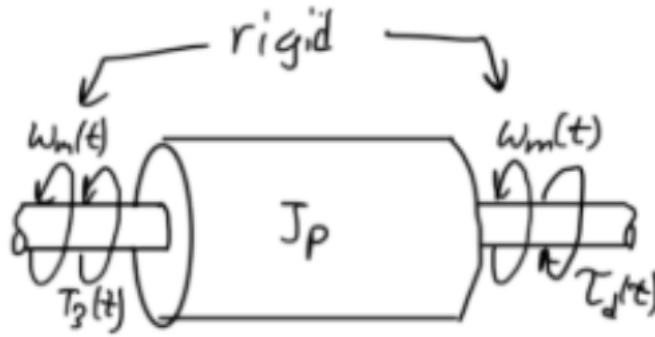
$$\sum T_\theta = T_t(t) - T_2(t) = J_m \dot{\omega}(t)$$

$$T_t(t) = T_2(t) + J_m \dot{\omega}(t)$$



$$\Sigma T_\theta = T_2(t) - T_3(t) = J_h \dot{\omega}_m(t)$$

$$T_2(t) = T_3(t) + J_h \dot{\omega}_m(t)$$



$$\Sigma T = T_3(t) - T_d(t) = J_p \dot{\omega}_m(t)$$

$$T_3(t) = T_d(t) + J_p \dot{\omega}_m(t)$$

$$(1) V_m(t) = R_m i_L(t) + L_m \frac{di_L(t)}{dt} + e_b(t)$$

$$(2) e_b(t) = k_m \omega_m(t)$$

$$(3) T_t(t) = i_L(t) k_t$$

$$(4) T_t(t) = T_d(t) + (J_p + J_h + J_m) \dot{\omega}_m(t)$$

$$(5) T_d(t) = k_d \omega_m(t)$$

$$(3) \& (5) \rightarrow (4)$$

$$k_t i_L(t) = k_d \omega_m(t) + (J_p + J_h + J_m) \dot{\omega}_m(t)$$

Solving for i_L and taking the derivative and plugging into (1)

$$V_m(t) = R_m \left[\frac{(J_p + J_h + J_m) \dot{\omega}_m(t)}{k_t} + \left(\frac{k_d}{k_t} \right) \omega_m(t) \right] + L_m \left[\frac{(J_p + J_h + J_m)}{k_t} \ddot{\omega}_m(t) + \left(\frac{k_d}{k_t} \right) \dot{\omega}_m(t) \right] + k_m \omega_m(t)$$

$$L_m \frac{(J_p + J_h + J_m)}{k_t} \ddot{\omega}_m(t) + \left[R_m \frac{(J_p + J_h + J_m)}{k_t} + \left(\frac{L_m k_d}{k_t} \right) \right] \dot{\omega}_m(t) + \left(\left(\frac{R_m k_d}{k_t} \right) + k_m \right) \omega_m(t) = V_m(t)$$

We arrive at:

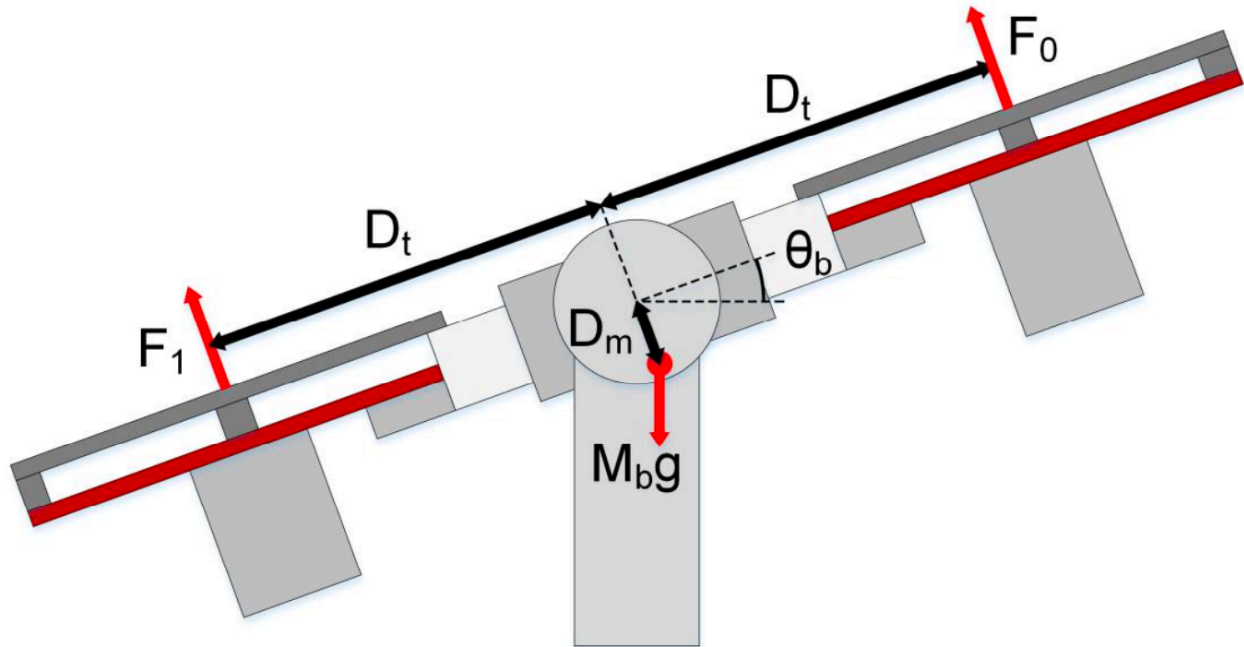
$$\ddot{\omega}_m(t) + \left[\frac{R_m}{L_m} + \frac{k_d}{J_p + J_h + J_m} \right] \dot{\omega}_m(t) + \left[\frac{R_m k_d + k_m k_t}{L_m (J_p + J_h + J_m)} \right] \omega_m(t) = \frac{k_t}{L_m (J_p + J_h + J_m)} V_m(t)$$

Plugging in values:

$$\boxed{\ddot{\omega}_m(t) + 7242.3 \dot{\omega}_m(t) + (1.4220 \times 10^5) \omega_m(t) = (3.2319 \times 10^6) V_m(t)}$$

Pitch Model

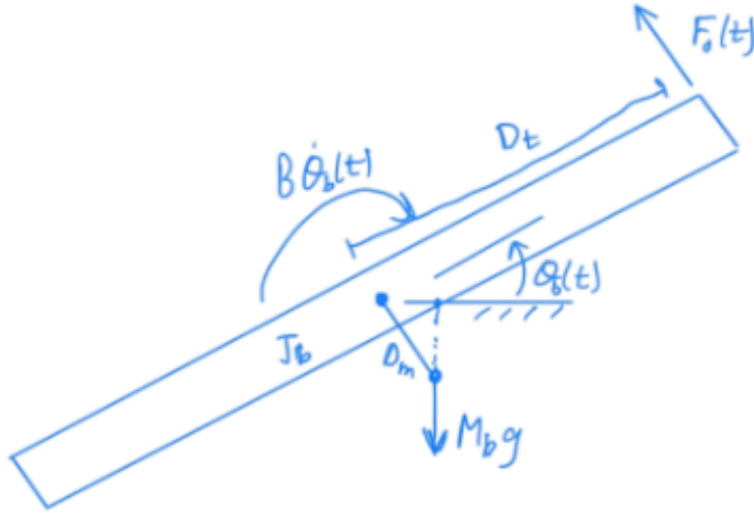
Figure 3 is a diagram of the Quanser Aero's pitch model found in the Quanser Aero workbook. For this project, our propeller 1 (left) will be pointing out of the page. This means that the vector of force F_1 (caused by the thrust from propeller 1) will be parallel to pitch rotation axis. Therefore, we will not consider force F_1 in the pitch model.



Symbol	Description	Value
M_b	Aero body mass	1.15 kg
D_t	Thrust displacement	0.158 m
D_m	Center of mass displacement	0.0071 m

Assumptions

- $\sin(\theta) \approx \theta$
- When modeling the pitch of the device, assume there is no angular velocity in the Yaw direction.
- Damping caused by friction is not negligible



$$\Sigma T_{\theta_b} = D_t F_0(t) - B \dot{\theta}_b(t) - (D_m \sin(\theta_b)) M_b g = J_b \ddot{\theta}_b(t)$$

$$\ddot{\theta}_b(t) + \left(\frac{B}{J_b}\right) \dot{\theta}_b(t) + \left(\frac{M_b g D_m}{J_b}\right) \theta_b(t) = \left(\frac{D_t}{J_b}\right) F_0(t)$$

$$\ddot{\theta}_b(t) + \left(\frac{B}{J_b}\right) \dot{\theta}_b(t) + \left(\frac{0.080099}{J_b}\right) \theta_b(t) = \left(\frac{0.158}{J_b}\right) F_0(t)$$

Pitch Model Identification

To determine the unknown variables, additional equations are required.

A technique is used to determine the unknowns by utilizing the system's transfer function and incorporating experimental data.

Transfer function → Assume I.C. are Zero

$$s^2 \theta_b(s) + s \cdot \frac{B}{J_b} \theta_b(s) + \frac{0.080099}{J_b} \theta_b(s) = \left(\frac{0.158}{J_b}\right) F_0(s)$$

$$\frac{\theta_b(s)}{F_0(s)} = \frac{\frac{0.158}{J_b}}{s^2 + \frac{B}{J_b} s + \frac{0.080099}{J_b}}$$

We know that the poles of a transfer function will give us $P_{1,2} = \sigma \pm j\omega$

$$P_{1,2} = -\frac{B}{2J_b} \pm \frac{\sqrt{B^2 - 4J_b(0.080099)}}{2J_b}$$

Given a $2 \times T$ matrix, the first row represents the time stamps, and the second row contains the angular displacement of the pitch angle recorded during the experiment. The initial condition is defined as $\theta_b(0) = 0.94$

Graphing the points in MATLAB shows that the system is underdamped, which can be modeled by the differential equation $\theta(t) = \theta_c(t) + \theta_p(t)$. For this experiment, the driving force is zero, so there is no particular solution. Therefore, we assume the form of the homogeneous solution:

$$\theta(t) = 2Ce^{\sigma t} \cos(\omega t)$$

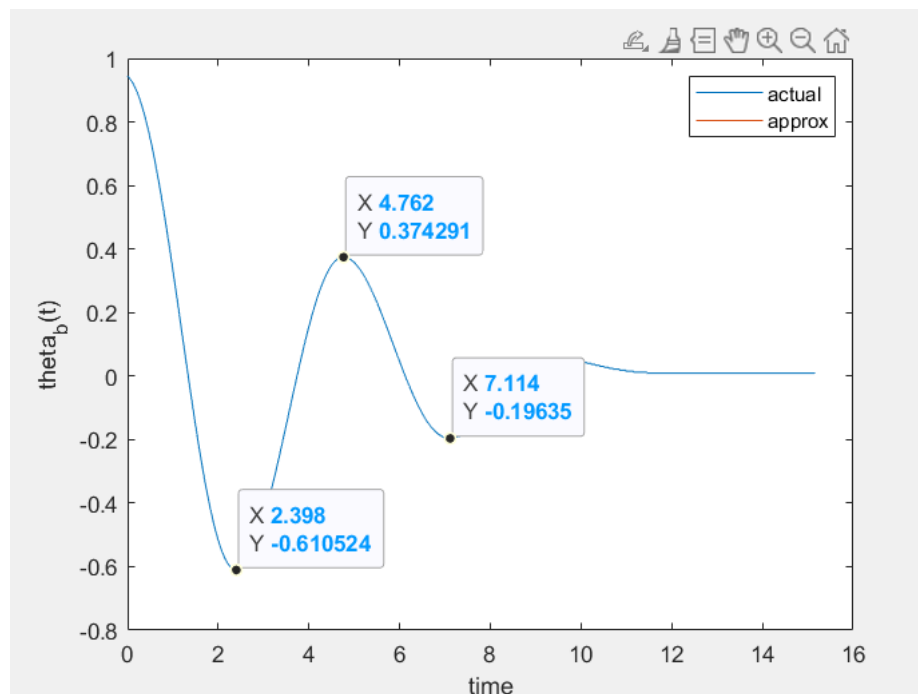
By estimating the period from the graph, we find it to be approximately 4.7630 seconds.

$$\omega = \frac{2\pi}{T} = \frac{2\pi}{4.7630} = 1.319 \frac{\text{rad}}{\text{s}}$$

Rearranging to solve for σ

$$\sigma = \frac{\ln\left(\frac{\theta(t)}{0.94 \cos(1.319t)}\right)}{t}$$

The estimate for σ is most accurate near the peaks, so several peaks were used to calculate its value. These individual σ values were then averaged to obtain the final estimated σ .

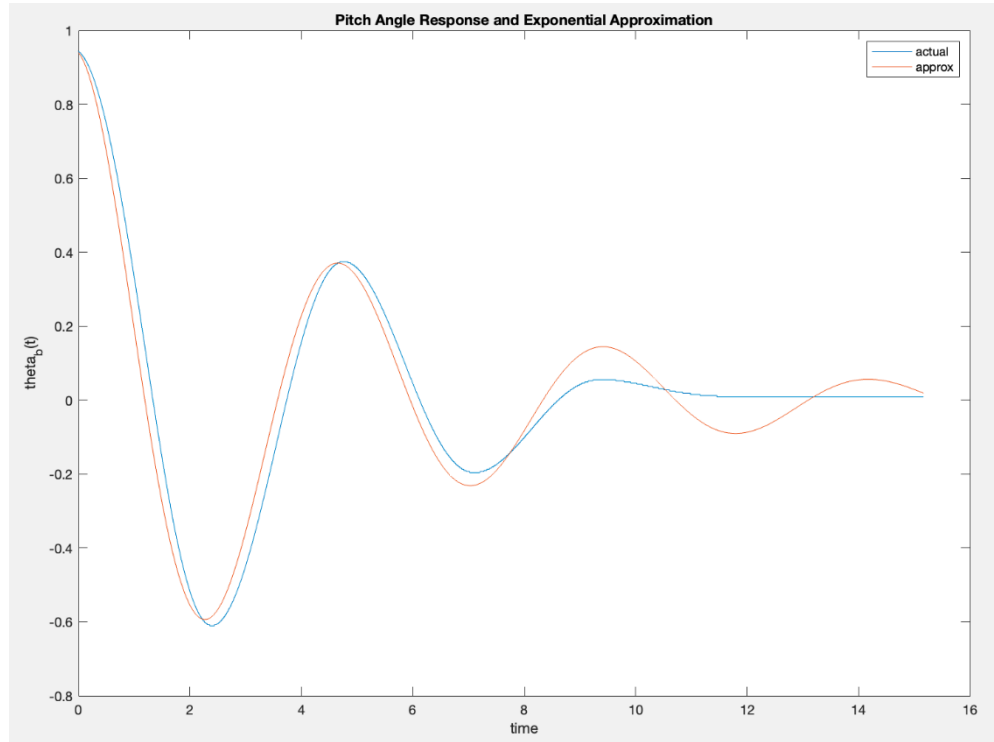


Sigma was estimated to be $\sigma = -0.1978$

Substituting the values, we arrive at the solution to our differential equation:

$$\theta_b(t) = 2Ce^{-0.1978t} \cos(1.319t)$$

Graphing this response vs experimental data



Now that we have estimated values for σ and ω we obtain two equations with two unknowns.

$$\sigma = -\frac{B}{2J_b} \quad J\omega = \frac{\sqrt{B^2 - 4J_b(0.080099)}}{2J_b}$$

Starting with the equation for ω , we can eliminate J as long as we switch the signs under the square root. $j\omega = \sqrt{A - B} \rightarrow \omega = \sqrt{B - A}$

$$\omega = \frac{\sqrt{4J_b(0.080099) - B^2}}{2J_b}$$

$$\omega^2 = \frac{-B^2 + 4J_b(0.080099)}{4J_b^2} \rightarrow \frac{-B^2}{4J_b^2} + \frac{4J_b(0.080099)}{4J_b^2}$$

$$\omega^2 = \frac{(0.080099)}{J_b} - \frac{B^2}{4J_b^2}$$

$$B^2 = 4J_b(0.080099) - 4\omega^2 J_b^2$$

Plugging in $B = -2\sigma J_b$

$$4\sigma^2 J_b^2 = 4J_b(0.080099) - 4\omega^2 J_b^2$$

$$(4\omega^2 + 4\sigma^2)J_b^2 - 4(0.080099)J_b = 0$$

Factoring J_b

$$J_b((4\omega^2 + 4\sigma^2)J_b - 4(0.080099)) = 0$$

Since we know $J_b \neq 0$, then $J_b = 0$ is not a valid root. Therefore, the equation simplifies to:

$$(4\omega^2 + 4\sigma^2)J_b - 4(0.080099) = 0$$

$$J_b = \frac{4(0.080099)}{4\omega^2 + 4\sigma^2}$$

Plugging in our values for σ and ω

$$J_b = \frac{4(0.080099)}{4(1.319 \frac{rad}{s})^2 + 4(-0.1978)^2} = 0.0450 \text{ kg} \cdot \text{m}^2$$

$$B = -2\sigma J_b = -2(-0.1978)(0.0450) = 0.0178 \text{ N} \cdot \text{m} \cdot \text{s}$$

We finally arrive at a differential equation that relates the pitch angle to the thruster force:

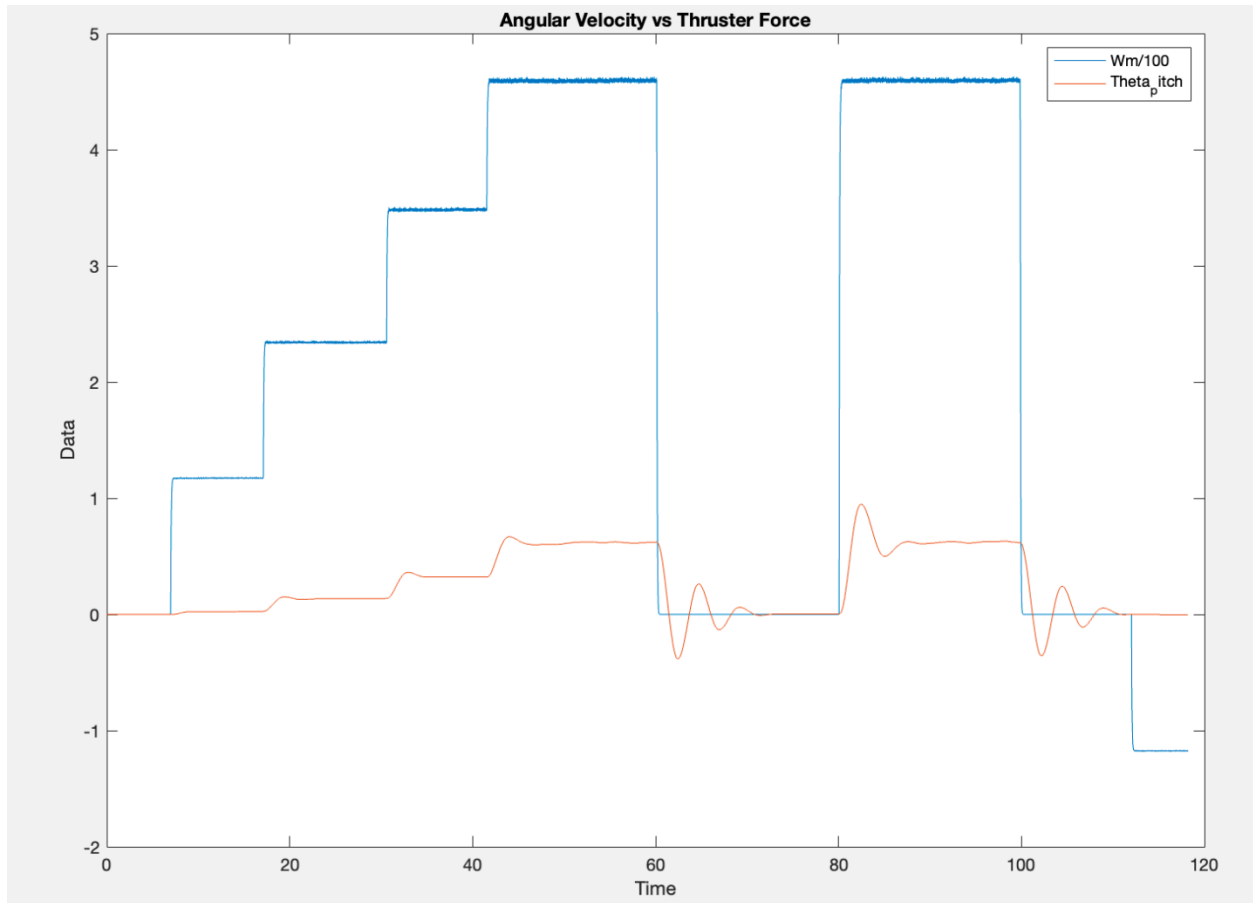
$$\boxed{\ddot{\theta}_b(t) + 0.3995\dot{\theta}_b(t) + 1.7793\theta_b(t) = 3.5098F_0(t)}$$

Thruster Force Identification

At this point, we've derived two differential equations: one describing the thruster's angular velocity with an input of applied motor voltage, and another describing the Aero's pitch angle with the thruster force as the input.

We are given a $3 \times T$ matrix: the first row contains the time steps, the second row is the thruster motor's angular velocity, and the third row is the Aero's pitch angle. Assume the function of the force w.r.t. voltage for both thrusters should be the same.

First, the thruster force from the given data is plotted in MATLAB to visualize the relationship between motor speed and the resulting thrust.



To derive the equation relating thruster force and angular velocity, we use the pitch equation derived earlier:

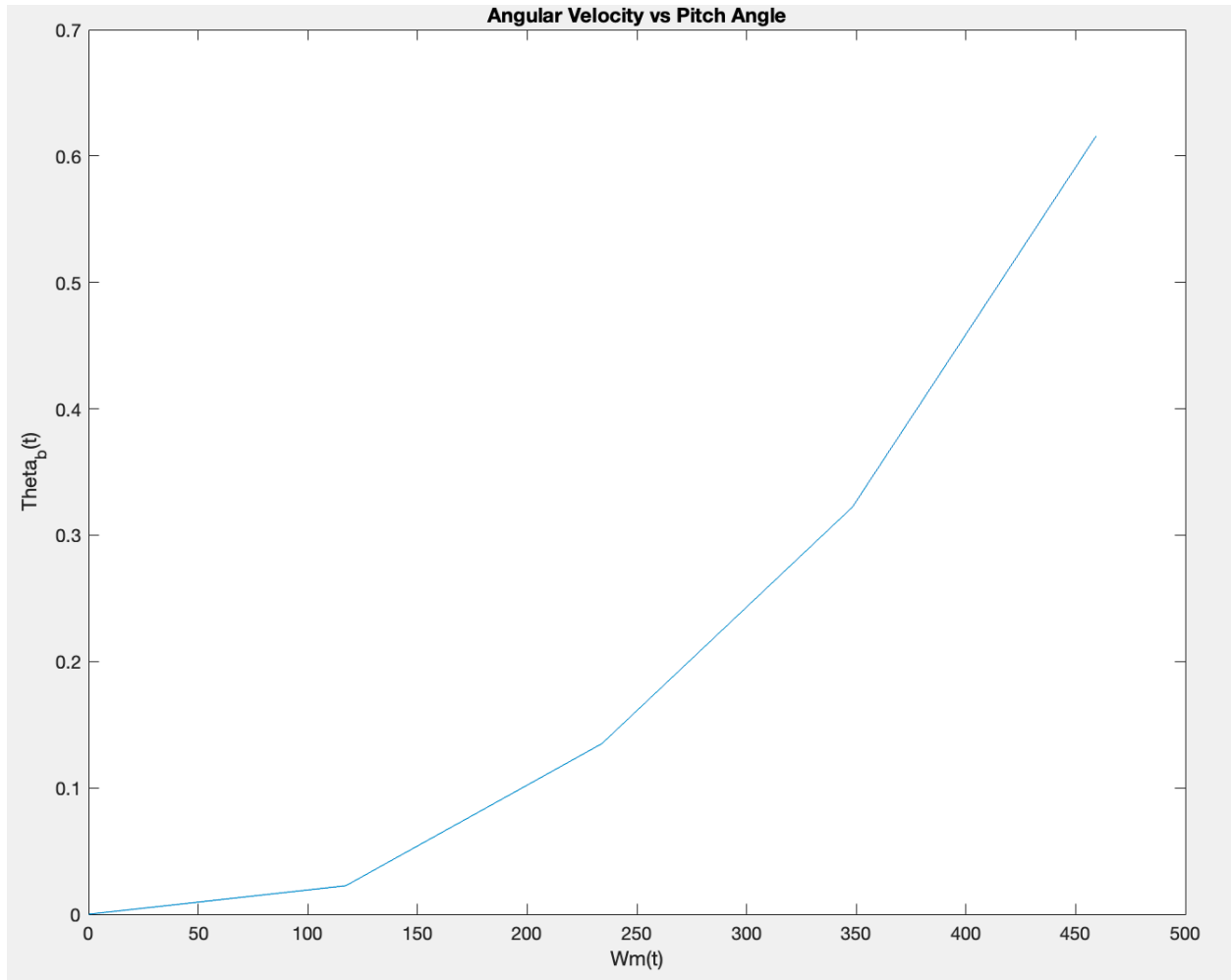
$$\ddot{\theta}_b(t) + 0.3995\dot{\theta}_b(t) + 1.7793\theta_b(t) = 3.5098F_0(t)$$

The graph shows us that the angular velocity and the pitch angle (θ_b) are related. To minimize error caused by oscillations in the experimental data, the equations are analyzed under steady-state conditions, where $\ddot{\theta}_b = 0$, $\dot{\theta}_b = 0$.

$$1.7793\theta_b(t) = 3.5098F_0(t)$$

$$\theta_b(t) = \frac{3.5098}{1.7793}F_0(t)$$

Using the data from experiential data $\omega_m(t)$ is graphed vs $\theta_b(t)$



The graph shows a quadratic relationship between $\omega_m(t)$ and $\theta_b(t)$ so it can be stated that:

$$\theta_b(t) = \frac{3.5098}{1.7793} F_0(t) = A\omega_m^2(t)$$

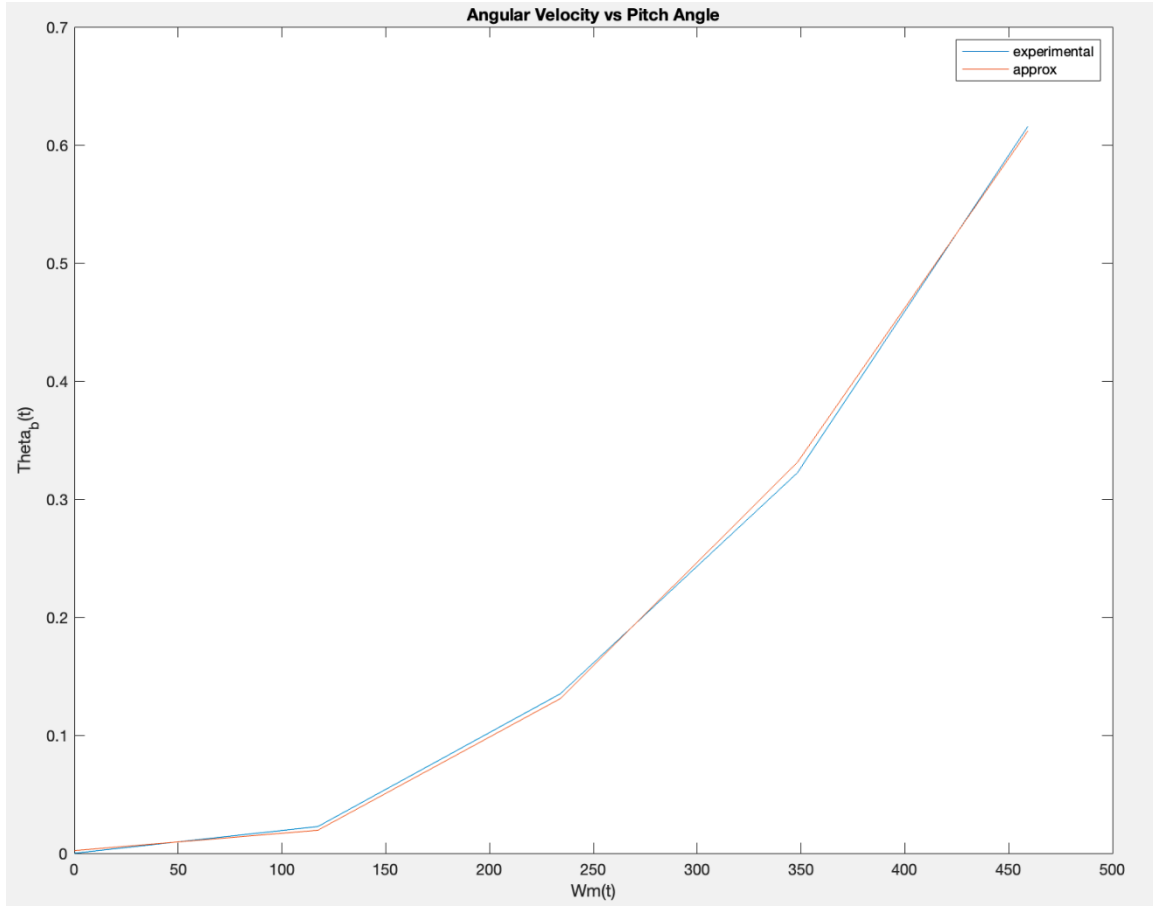
To achieve the most accurate model, the full quadratic expression is used.

$$\theta_b(t) = \frac{3.5098}{1.7793} F_0(t) = A\omega_m^2(t) + B\omega_m(t) + C$$

Using MATLAB, the polyfit function was applied to determine the quadratic coefficients that best fit the experimental data from the graph. The values were as follows:

$$A = 3.4522 \times 10^{-6} \quad B = -2.5805 \times 10^{-4} \quad C = 0.0022$$

Graphing this function with the experimental data shows that the function is fairly accurate to the data:



With the coefficients determined, we can now rearrange for $F_0(t)$, arriving at an equation that relates the thrust force to the angular velocity of the fan.

$$F_0(t) = \frac{1.7793}{3.5098} [A\omega_m^2(t) + B\omega_m(t) + C]$$

To simplify the equation, new constants are defined.

$$X = A \frac{1.7793}{3.5098}, \quad Y = B \frac{1.7793}{3.5098}, \quad Z = C \frac{1.7793}{3.5098}$$

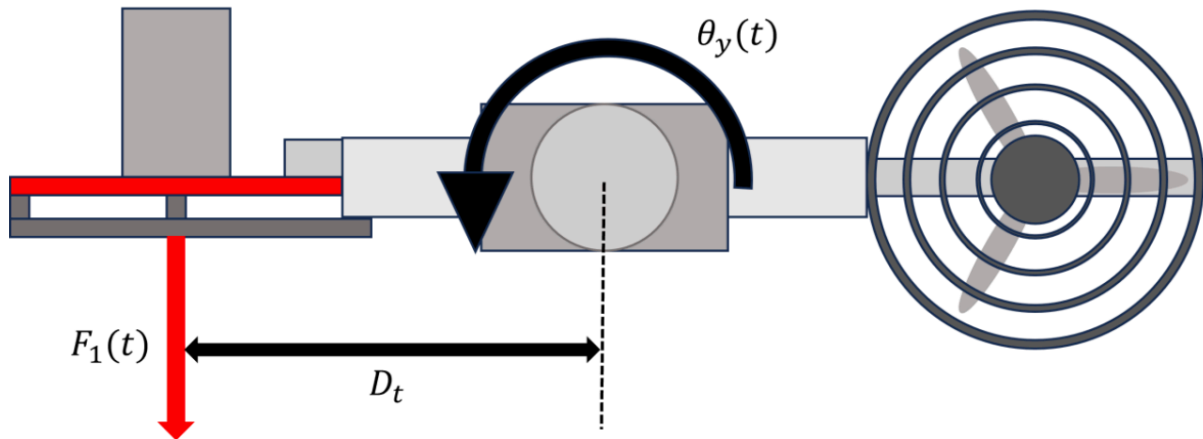
$$F_0(t) = X\omega_m^2(t) + Y\omega_m(t) + Z$$

By substituting our values, we obtain a function that relates the thrust force to the angular velocity of the fan.

$$F_0(t) = (1.7501 \times 10^{-6})\omega_m^2(t) - (1.3082 \times 10^{-4})\omega_m(t) + 0.0011$$

Yaw Model

This is diagram of the yaw model from a top view of the Quanser Aero. This is the correct configuration for the direction in which the fans will be pointed.



There are some unknown values for parameters of this model that we will solve for in the next project.

Assumptions

- The pitch angle of the Aero is allowed to move and be in various orientations.
- Force caused by the thruster 1 $F_1(t)$ is not affected by Yaw angular velocity or position.
- The function of the force w.r.t. voltage for both thrusters should be the same.
- Damping caused by friction is **not** negligible.

Derive the the yaw model as a single-input multi-output differential equation. Let the yaw angle $\theta_y(t)$ and pitch angle $\theta_b(t)$ be the outputs.

$$\Sigma T_{\theta_y} \rightarrow F_1(t)D_t \cos(\theta_b(t)) - T_c \operatorname{sgn}(\Omega_y(t)) - B\Omega_y(t) = J_y(\theta_b(t))\dot{\Omega}_y(t)$$

MATLAB Code

```
Rm = 8.4;      % ohms    Armature Resistance
Lm = 0.00116;  % H       Armature inductance
Kd = 1 * 10^(-5); % N*m/(rad/s) air resistance constant
Kt = 0.042;    % N*m/A    Torque Constant
Km = 0.042;    % V/(rad/s) Motor back-emf constant
Jp = 7.2 * 10^(-6); % kg*m^2 propeller inertia
Jh = 3.04 * 10^(-9); % kg*m^2 propeller hub inertia
```

```

Jm = 4 * 10^(-6); % kg*m^2 rotor inertia
% Wm''(t) + a*Wm'(t) + b*Wm(t) = c*Vm(t)
a = Rm/Lm + Kd/(Jp + Jh + Jm);
b = (Rm*Kd + Kt*Km)/(Lm*(Jp + Jh + Jm));
c = Kt/(Lm*(Jp + Jh + Jm));
%% pitch data
pitch_data;
t = pitch_data(1, :);
angular_displacement = pitch_data(2, :);
%% finding sigma
% since we are approximating the time period of the pitch data there will
% be values near the x axis where our cos will give us neg/pos and the
% angular displacement will have the opposite sign. Additionally, as
% angular displacement approaches the x_axis the error due to linearizing
% the differential equation spikes. Therefore to gather the most accurate
% values for sigma we will only look at the peaks of the angular
% displacement graph to calculate sigma.
T_range = 4.690:0.002:4.836; % Time values at max amplitude for first peak
T = mean(T_range);
omega_b = 2*pi/T; % Approximation from graph
% includes all values of sigma including complex numbers
sigma = log(angular_displacement./((0.94.*cos(omega_b.*t)))./t);
% using the time stamps for the peaks in angular displacement we can
% find the corresponding sigma values at those peaks (Note that since this
% is experimental data a small range of time was required for the program
% to actually select that time stamp).
time_of_peaks = (((t > 2.397)&(t < 2.399))|((t > 4.761)&(t < 4.763))|((t > 7.113)&(t < 7.115)));
sigma_at_peaks = sigma(time_of_peaks);
% the average of those sigma values
avg_sigma = mean(sigma_at_peaks);
% plotting the pitch angle response with average sigma to verify that it
% correctly approximates the decay
appxfunction = 0.94.*exp(avg_sigma.*t).*cos(omega_b.*t);
figure();
plot(t, angular_displacement);
hold('on');
xlabel("time");
ylabel("theta_b(t)");
plot(t, appxfunction);
legend("actual", "approx");
title("Pitch Angle Response and Exponential Approximation");
%% solving for B and Jb
Jb = (0.080099)./(omega_b^2+avg_sigma^2);
B = -avg_sigma.*2.*Jb;
%% Pitch model coefficients
% Th''(t) + d*Th'(t) + e*Th(t) = f*F_0(t)
Mb = 1.15; % kg (aero mass)
Dt = 0.158; % m (distance to thruster from pivot point)
Dm = 0.0071; % m (center of mass displacement)
gravity = 9.81; % m/s^2
d = B/Jb;
e = Mb*gravity*Dm/Jb;
f = Dt/Jb;
%% finding relationship between fan speed and force
thruster_force_exp = data;
t2 = thruster_force_exp(1, :);

```

```

Wm = thruster_force_exp(2, :);
Theta_pitch = thruster_force_exp(3, :);
figure();
plot(t2, (Wm./100));
hold("on");
plot(t2, Theta_pitch);
xlabel("Time");
ylabel("Data");
legend("Wm/100", "Theta_pitch");
title("Angular Velocity vs Thruster Force")
%% Finding thruster function
% times when the angle reaches steady state
ss1 = (t2 < 5);
ss2 = ((t2 > 11) & (t2 < 16));
ss3 = ((t2 > 23) & (t2 < 29));
ss4 = ((t2 > 36) & (t2 < 40));
ss5 = ((t2 > 48) & (t2 < 59));
Th_b_1 = mean(Theta_pitch(ss1));
Th_b_2 = mean(Theta_pitch(ss2));
Th_b_3 = mean(Theta_pitch(ss3));
Th_b_4 = mean(Theta_pitch(ss4));
Th_b_5 = mean(Theta_pitch(ss5));
Wm1 = mean(Wm(ss1));
Wm2 = mean(Wm(ss2));
Wm3 = mean(Wm(ss3));
Wm4 = mean(Wm(ss4));
Wm5 = mean(Wm(ss5));
Wm_ss = [Wm1, Wm2, Wm3, Wm4, Wm5];
Th_b = [Th_b_1, Th_b_2, Th_b_3, Th_b_4, Th_b_5];
figure();
plot(Wm_ss, Th_b);
xlabel("Wm(t)");
ylabel("Theta_b(t)");
title("Angular Velocity vs Pitch Angle");
% we can see that the relationship between Wm and theta_b is parabolic
% we will assume the form
% theta_b(t)_ss = x*Wm^2(t) + y*Wm(t) + z
polynomial = polyfit(Wm_ss, Th_b, 2);
x = polynomial(1);
y = polynomial(2);
z = polynomial(3);
% since  $F_0 = e/f * \Theta_b(t)$  at ss we can find
%  $F_0 = X*(Wm(t))^2 + Y*Wm(t) + Z$  where:
X = x*e/f;
Y = y*e/f;
Z = z*e/f;
% to see how accurate the function is to the thruster data...
F_0_check = polyval(polynomial, Wm_ss);
hold("on");
plot(Wm_ss, F_0_check);
legend("experimental", "approx");

```

Project Part 2

Thruster Model Validation

To verify that our derived thruster model matches the real system behavior, we used the provided experimental data `thruster_exp_data.mat`. This file contains time, applied motor voltage, and measured propeller angular velocity.

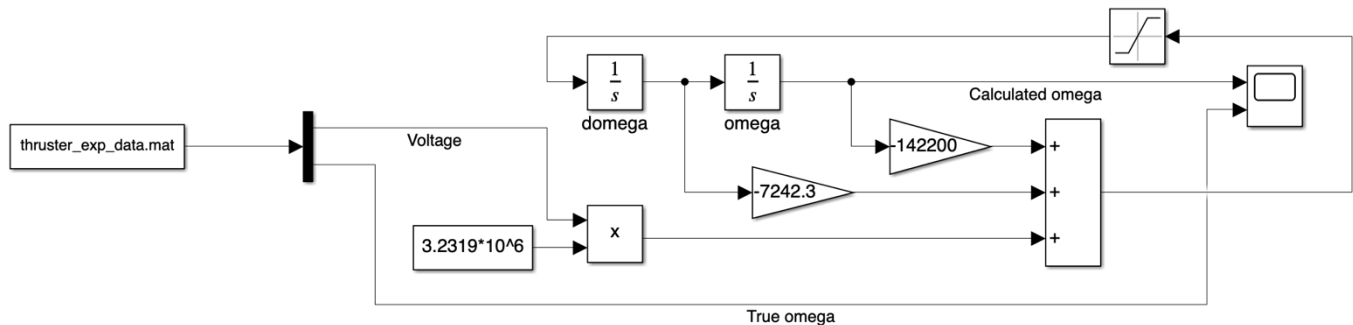
We created a Simulink model that reads this data using a From File block and separates the voltage and velocity signals with a Demux block. The voltage signal was used as the input to our thruster model, and the model was initialized to match the starting conditions in the data.

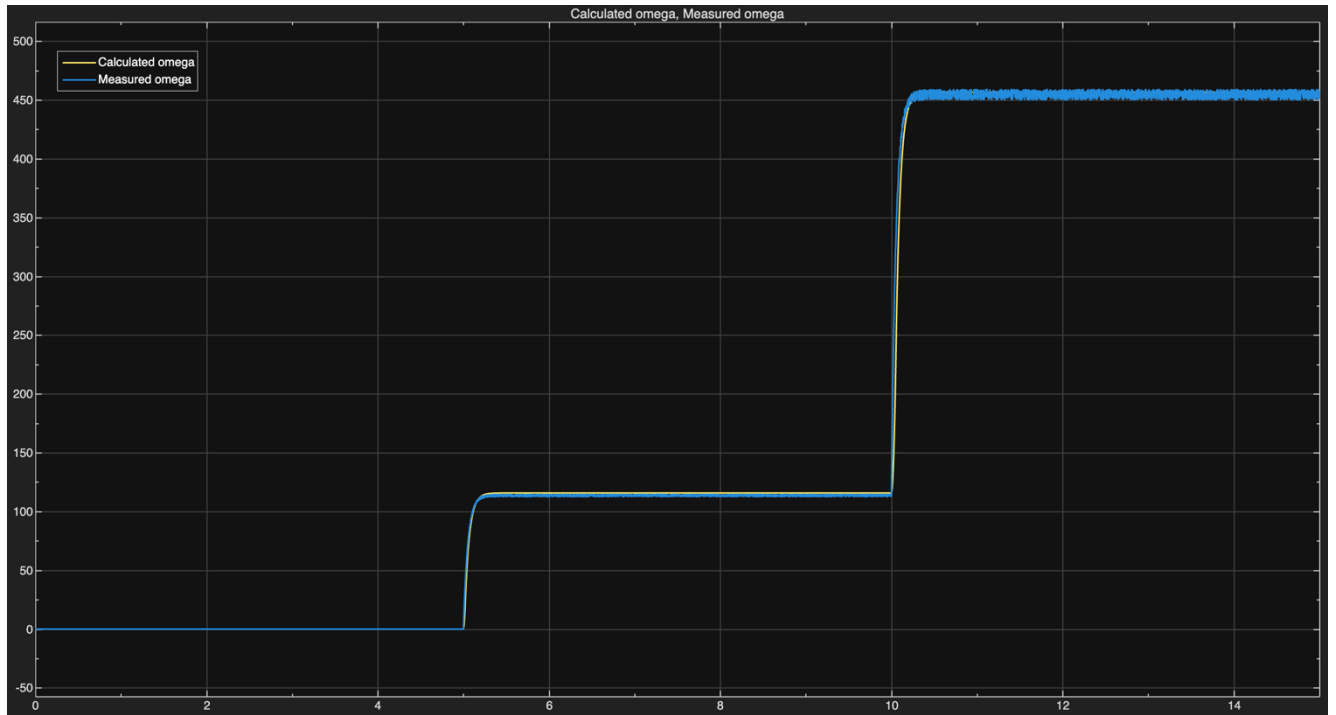
We then ran the simulation using the same input voltage as the experiment and compared the simulated output to the measured propeller velocity. Finally, we plotted both responses on the same graph to evaluate how well the model captured the real system behavior.

Previously derived Thruster Model:

$$\ddot{\omega}_m(t) + 7242.3\dot{\omega}_m(t) + (1.4220 \times 10^5)\omega_m(t) = (3.2319 \times 10^6)V_m(t)$$

The simulation is as follows:





Pitch Model System Identification

First, we validated the derived model that relates the pitch angle to the thruster force:

$$\ddot{\theta}_b(t) + 0.3995\dot{\theta}_b(t) + 1.7793\theta_b(t) = 3.5098F_0(t)$$

To do so, we need the model relating thrust force to the fan's angular velocity.

$$F_0(t) = (1.7501 \times 10^{-6})\omega_m^2(t) - (1.3082 \times 10^{-4})\omega_m(t) + 0.0011$$

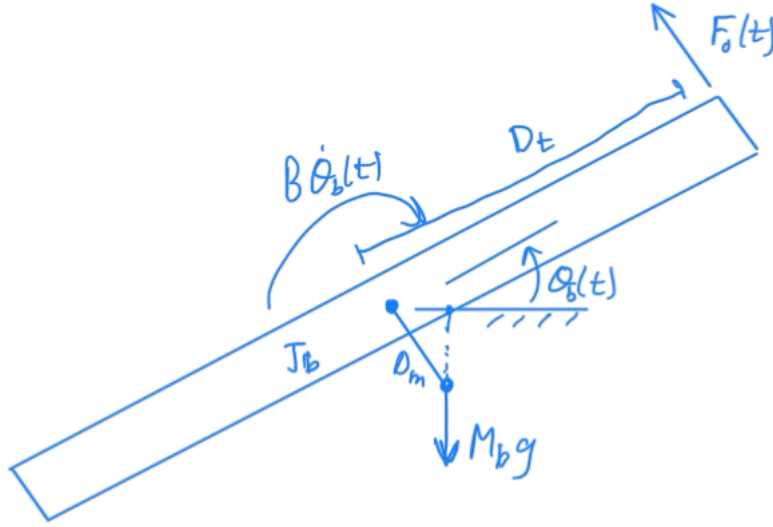
We are given “thruster_force_exp.mat” data. This data includes the propeller's angular velocity and pitch angle.

Using the given angular velocity as the input and the pitch angle as the output from our previously derived model, the pitch angle is plotted vs the true pitch angle.

Next the pitch model is calculated with a new definition of torque caused by friction.

$$T_f(t) = T_c \text{sgn}(\Omega(t)) + B\Omega(t)$$

Also, this model will not be linearized, so the pitch angle will be based on $\sin(\theta_b(t))$.



$$\Sigma T_{\theta_b} = D_t F_0(t) - T_f - (D_m \sin(\theta_b)) M_b g = J_b \ddot{\theta}_b(t)$$

$$J_b \ddot{\theta}_b(t) = -B \dot{\theta}_b(t) - T_c \operatorname{sgn}(\dot{\theta}_b(t)) - D_m M_b g \sin(\theta_b(t)) + D_t F_0(t)$$

$$\ddot{\theta}_b(t) = -\frac{B}{J_b} \dot{\theta}_b(t) - \frac{T_c}{J_b} \operatorname{sgn}(\dot{\theta}_b(t)) - \frac{D_m M_b g}{J_b} \sin(\theta_b(t)) + \frac{D_t}{J_b} F_0(t)$$

We now arrive at a differential equation relating pitch angle and thruster force, but all the constants are unknown. Since we are given experimental data for pitch angle and angular velocity, we can discretize and use those values to approximately solve for the unknown coefficients. The unknown coefficients are grouped as follows.

$$\ddot{\theta}_b(t) = K_1 \dot{\theta}_b(t) + K_2 \operatorname{sgn}(\dot{\theta}_b(t)) + K_3 \sin(\theta_b(t)) + K_4 F_0(t)$$

To discretize, we use the following equations.

$$\ddot{\theta}_b(t) = \frac{\theta_b(n+1) - 2\theta_b(n) + \theta_b(n-1)}{T_s^2} \quad \dot{\theta}_b = \frac{\theta_b(n) - \theta_b(n-1)}{T_s}$$

$$\frac{\theta_b(n+1) - 2\theta_b(n) + \theta_b(n-1)}{T_s^2} = K_1 \left(\frac{\theta_b(n) - \theta_b(n-1)}{T_s} \right) + K_2 \left(\operatorname{sgn} \left(\frac{\theta_b(n) - \theta_b(n-1)}{T_s} \right) \right) + K_3 (\sin(\theta_b(n))) + K_4 (F_0(n))$$

To make our lives easier, the equation is put in a matrix form.

$$\vec{K} = [K_1 \quad K_2 \quad K_3 \quad K_4] \quad \vec{s}_n = \begin{bmatrix} \frac{\theta_b(n) - \theta_b(n-1)}{T_s} \\ \operatorname{sgn}\left(\frac{\theta_b(n) - \theta_b(n-1)}{T_s}\right) \\ \sin(\theta_b(n)) \\ F_0(n) \end{bmatrix}$$

We arrive at a difference equation relating pitch angle and thruster force

$$\theta_b(n+1) = 2\theta_b(n) - \theta_b(n-1) + T_s^2 \vec{K} \vec{s}_n$$

To solve for the unknown values of $\vec{K}$, the cost function is implemented.

$$J = \sum_{n=0}^{N-2} \frac{1}{2} (\text{Estimate} - \text{true})^2$$

When the cost function is minimized, the estimate of $\vec{K}$ becomes the true $\vec{K}$.

Plugging the difference equation into the cost function and taking the partial derivative with respect to $\vec{K}$, we arrive at the cost function with respect to $\vec{K}$. Setting the partial derivative equal to zero, we can solve for our unknown vector $\vec{K}$.

$$J = \sum_{n=0}^{N-2} \frac{1}{2} (2\theta_b(n) - \theta_b(n-1) + T_s^2 \vec{K} \vec{s}_n - \theta_b(n+1))^2$$

$$\frac{\partial J}{\partial \vec{K}} = \{2\theta_b(n) - \theta_b(n-1) + T_s^2 \vec{K} \vec{s}_n - \theta_b(n+1)\} T_s^2 \vec{s}_n^T = [0 \quad 0 \quad 0 \quad 0]$$

To make our lives easier, $\vec{s}^T$ is distributed through.

$$\frac{\partial J}{\partial \vec{K}} = \{(2\theta_b(n) - \theta_b(n-1) - \theta_b(n+1)) \vec{s}_n^T + T_s^2 \vec{K} \vec{s}_n \vec{s}_n^T\} = [0 \quad 0 \quad 0 \quad 0]$$

$$T_s^2 \vec{K} \vec{s}_n \vec{s}_n^T = -(2\theta_{b(n)} - \theta_{b(n-1)} - \theta_{b(n+1)}) \vec{s}_n^T$$

We arrive at a solution for $\vec{K}$

$$\boxed{\vec{K} = \frac{1}{T_s^2} (-2\theta_{b(n)} + \theta_{b(n-1)} + \theta_{b(n+1)}) \vec{s}_n^T (\vec{s}_n \vec{s}_n^T)^{-1}}$$

To solve for the closed-form solution, we first need to define some vectors and matrices to solve for $\vec{K}$.

$$\vec{\theta}_{n-1} = [\theta(0) \quad \theta(1) \quad \dots \quad \theta(N-2)]^{1 \times N-1}$$

$$\vec{\theta}_n = [\theta(1) \quad \theta(2) \quad \dots \quad \theta(N-1)]^{1 \times N-1}$$

$$\vec{\theta}_{n+1} = [\theta(2) \quad \theta(3) \quad \dots \quad \theta(N)]^{1 \times N-1}$$

$$\vec{s}_n = [\vec{s}(1) \quad \vec{s}(2) \quad \dots \quad \vec{s}(N-1)]^{4 \times N-1}$$

$$\vec{F}_n = [\vec{F}(1) \quad \vec{F}(2) \quad \dots \quad \vec{F}(N-1)]^{1 \times N-1}$$

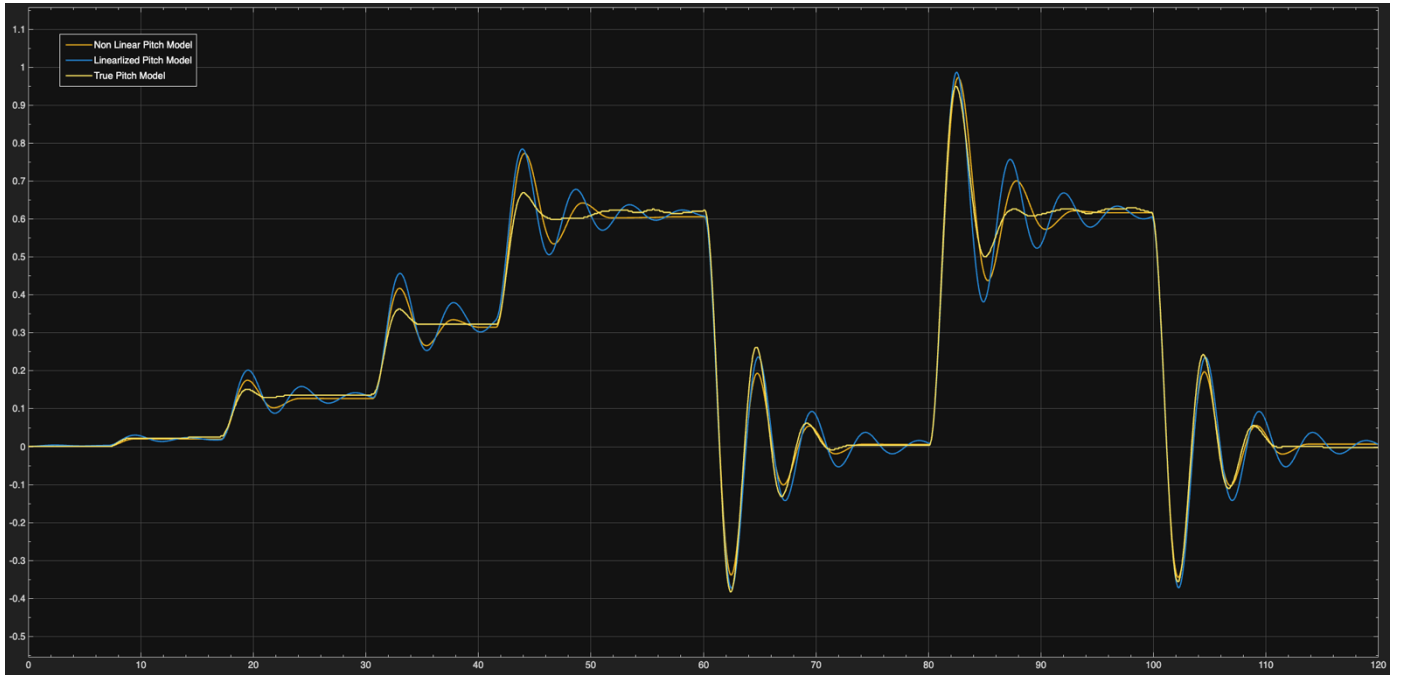
The solution was calculated in MATLAB, but first, to ensure the most accurate results, a lowpass filter was implemented to the given data, θ_b and F_0 . The results are as follows.

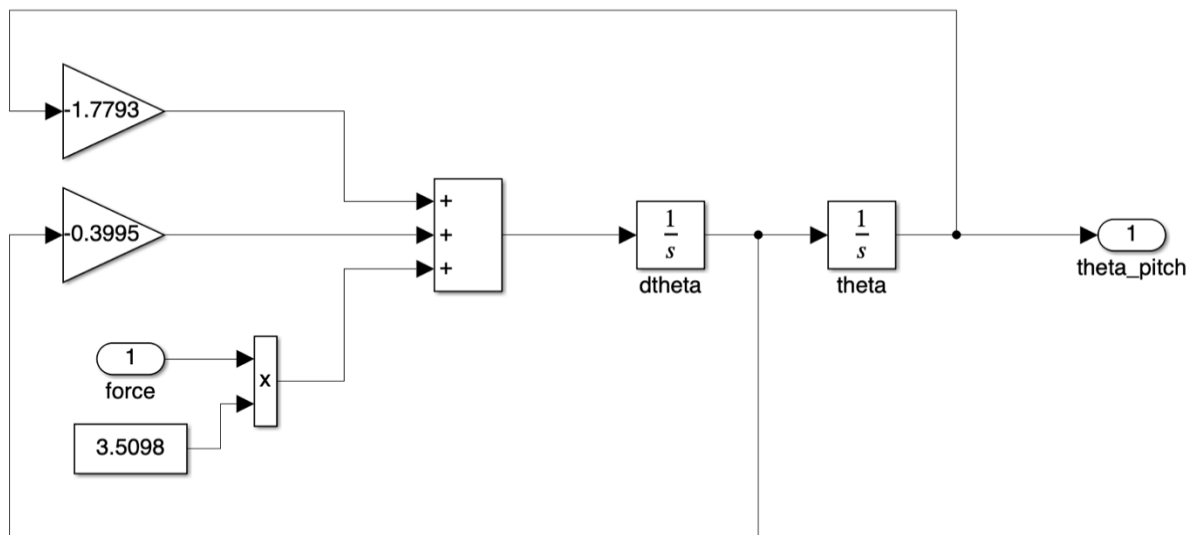
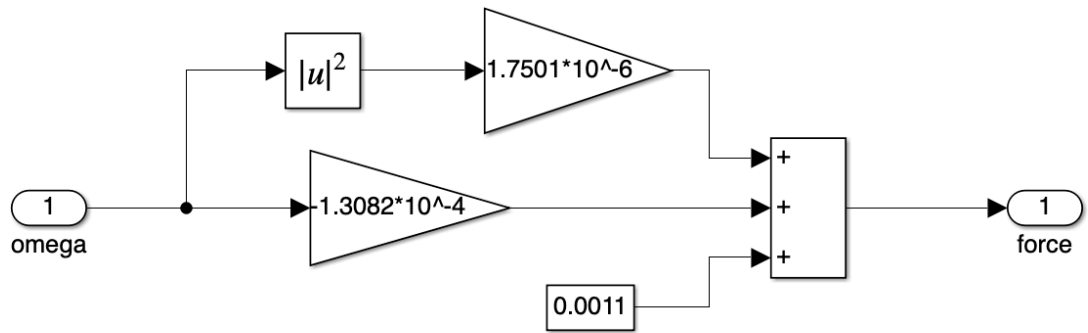
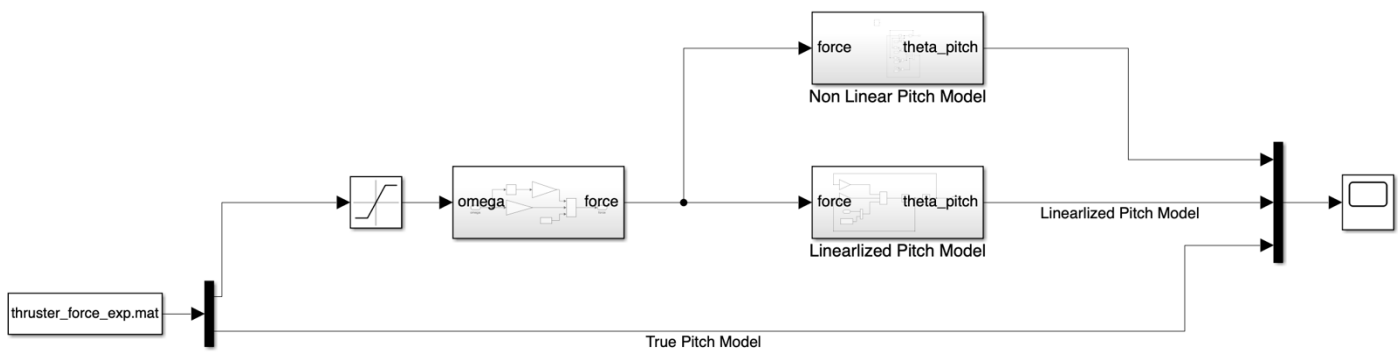
$$\vec{K} = [-0.4576 \quad -0.0108 \quad -1.9009 \quad 3.5112]$$

Finally, we can fully describe our nonlinear pitch model.

$$\ddot{\theta}_b(t) = -0.4576\dot{\theta}_b(t) - 0.0108\text{sgn}(\dot{\theta}_b(t)) - 1.9009\sin(\theta_b(t)) + 3.5112F_0(t)$$

In Simulink, the true pitch model was plotted against the linear and nonlinear pitch models. The simulation is as follows





First, we start with our derived model for yaw angular velocity, using the force from thruster 1 as the input.

$$F_1(t)D_t \cos(\theta_b(t)) - T_c \operatorname{sgn}(\Omega_y(t)) - B\Omega_y(t) = J_y(\theta_b(t))\dot{\Omega}_y(t)$$

The model has many unknowns that we must solve for, much like how we did the pitch model. Here, $J_y(\theta_b(t))$ is the moment of inertia of the yaw as a function of the pitch angle. This slightly complicates our model. As before, we lump the coefficients together as $\vec{K}$, but in this case, our coefficients are no longer constant because of $J_y(\theta_b(t))$. To solve this, we must find the constants at each pitch angle. In our measured data, we have 7 time steps with a constant pitch angle, which we will use to find 7 different $\vec{K}$. Using the values of K at each pitch angle, we can find the relationship between K_1 , K_2 , K_3 and θ_b .

Like before, in our pitch model, our first steps will be to lump coefficients, then discretize.

$$\dot{\Omega}_y(t) = \frac{D_t \cos(\theta_b(t))}{J_y(\theta_b(t))} F_1(t) - \frac{T_c \operatorname{sgn}(\Omega_y(t))}{J_y(\theta_b(t))} - \frac{B}{J_y(\theta_b(t))} \Omega_y(t)$$

$$\dot{\Omega}_y(t) = K_1 F_1(t) + K_2 \operatorname{sgn}(\Omega_y(t)) + K_3 \Omega_y(t)$$

To discretize the equations, we have the following.

$$\dot{\Omega}_y(t) = \frac{\Omega_y(n+1) - \Omega_y(n)}{T_s}$$

Substituting and solving for $\Omega_y(n+1)$, we arrive at.

$$\frac{\Omega_y(n+1) - \Omega_y(n)}{T_s} = K_1 F_1(t) + K_2 \operatorname{sgn}(\Omega_y(t)) + K_3 \Omega_y(t)$$

$$\Omega_y(n+1) = \Omega_y(n) + T_s K_1 F_1(t) + T_s K_2 \operatorname{sgn}(\Omega_y(t)) + T_s K_3 \Omega_y(t)$$

To make our lives easier, the equation is put in a matrix form.

$$\vec{K} = [K_1 \quad K_2 \quad K_3] \quad \vec{s}_n = \begin{bmatrix} F_1(t) \\ \operatorname{sgn}(\Omega_y(t)) \\ \Omega_y(t) \end{bmatrix}$$

$$\Omega_y(n+1) = \Omega_y(n) + T_s \vec{K} \vec{s}_n$$

As before, the cost function is minimized to solve for the unknown vector $\vec{K}$.

$$J = \sum_{n=0}^{N-2} \frac{1}{2} \left(\Omega_y(n) + T_s \vec{K} \vec{s}_n - \Omega_y(n+1) \right)^2$$

Taking the partial derivative and setting it equal to zero.

$$\frac{\partial J}{\partial \vec{K}} = \{ \Omega_y(n) + T_s \vec{K} \vec{s}_n - \Omega_y(n+1) \} T_s \vec{s}_n^T = [0 \quad 0 \quad 0 \quad 0]$$

Distributing $\vec{s}_n^T$ and Rearranging to solve for $\vec{K}$, we arrive at a solution for $\vec{K}$.

$$\vec{K} = \frac{1}{T_s} (\vec{\Omega}_{y(n+1)} - \vec{\Omega}_{y(n)}) \vec{s}_n^T (\vec{s}_n \vec{s}_n^T)^{-1}$$

Note, this equation is only true for values of Ω_y with a constant pitch angle θ_b . In MATLAB, we evaluate the above equation for $\vec{K}$, seven different times, each time with a new constant pitch angle and its corresponding $\vec{\Omega}_y$ values. We arrive at a $\vec{K}$ matrix where the columns are K_1 , K_2 , and K_3 respectively, while the rows are the coefficients for each constant pitch angle. With these constants, we can find the relationship with pitch angle to approximately find a function for $K_1(\theta_b(t))$, $K_2(\theta_b(t))$, and $K_3(\theta_b(t))$. The MATLAB function polyfit was used to find the relationship. A quadratic was used as the model. The equations are as follows.

$$K_1(\theta_b(t)) = -4.233424\theta^2 - 0.043935\theta + 5.996774$$

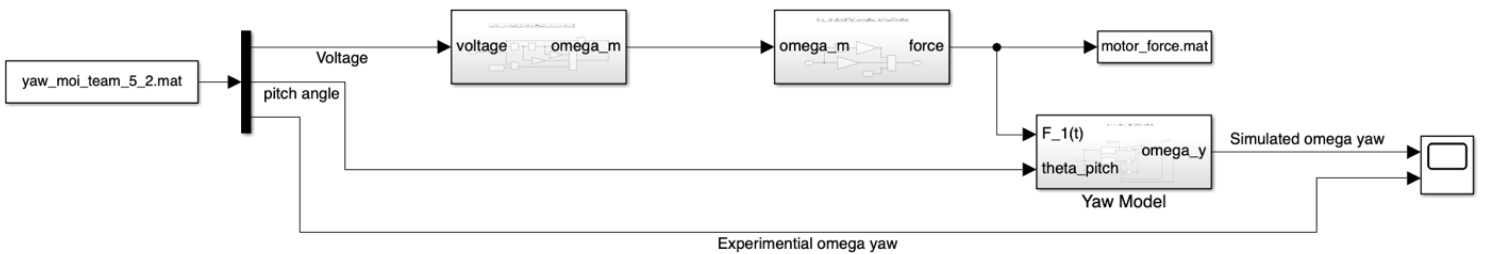
$$K_2(\theta_b(t)) = 0.204306\theta^2 - 0.023560\theta - 0.176635$$

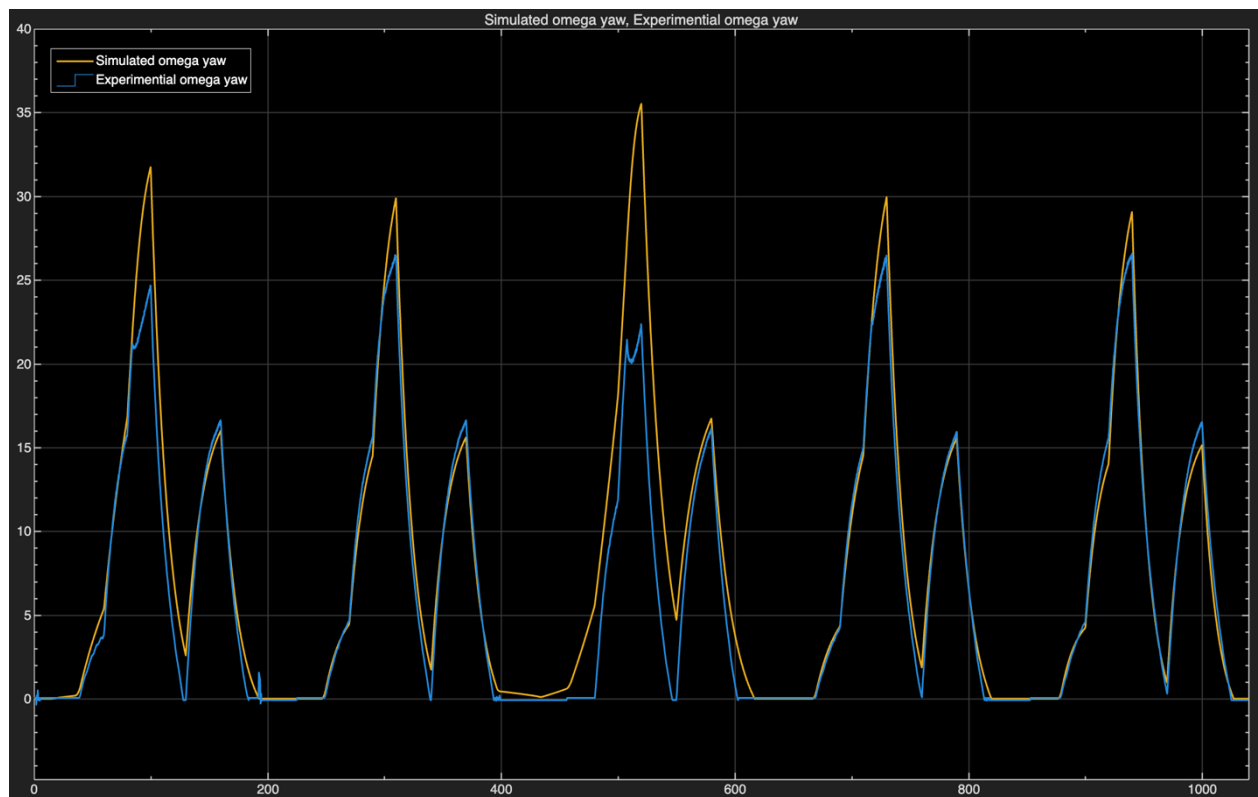
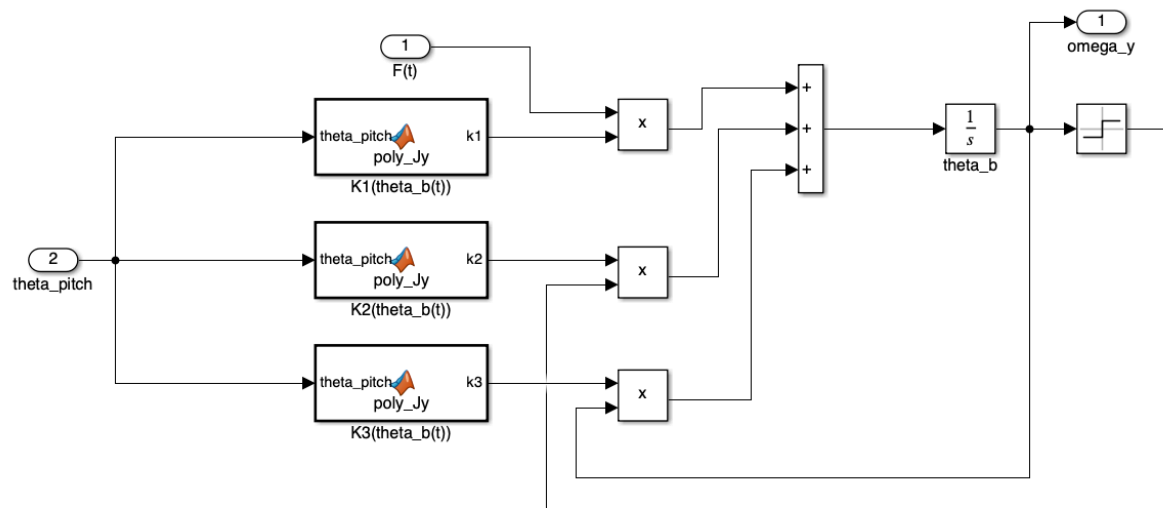
$$K_3(\theta_b(t)) = 0.096505\theta^2 + 0.000506\theta - 0.076792$$

With these new relationships, we can redefine our lumped coefficients model.

$$\dot{\Omega}_y(t) = K_1(\theta_b(t))F_1(t) + K_2(\theta_b(t))sgn(\Omega_y(t)) + K_3(\theta_b(t))\Omega_y(t)$$

To test this model, it is implemented in Simulink. The results are as follows.





The simulation shows an overshoot in yaw angular velocity at high speeds, which matches our expectations. This occurs because our original model doesn't account for several real-world factors that become very significant at high speeds, such as aerodynamic drag and

inherent system limitations. Also, motor efficiency decreases substantially at higher operating speeds. Overall, this model is a very good representation of our system.

Code

```
% Project 2 Code – Team 05
```

```
%% Pitch Model
```

```
% This file contains a  $3 \times T$  matrix where the first row is the time stamp of  
an experiment,  
% the second row is the angular velocity of the thruster's motor, and the  
third row is the  
% pitch angle of the Aero.  
load('thruster_force_exp.mat')
```

```
Ts = 0.002;  
omega = data(2,2:(end-1));  
F0 = 1.7501.*10^(-6).*omega.^2 + -1.308*10^(-4).*omega + 0.0011;
```

```
Fs = 1/Ts;      % sampling frequency (500 Hz)  
Fc = 10;        % cutoff frequency (Hz)  
N = 2;          % filter order 2
```

```
% Low-pass filter  
[b, a] = butter(N, Fc/(Fs/2));
```

```
% Filtered signals  
theta_filt = filtfilt(b, a, data(3,:));  
F0_filt = filtfilt(b, a, F0);
```

```
th_n = theta_filt(2:(end-1)); % theta(n)  
th_nm1 = theta_filt(1:(end-2)); % theta(n-1)  
th_np1 = theta_filt(3:(end)); % theta(n+1)  
s_n = [(th_n-th_nm1)./Ts; sign((th_n-th_nm1)./Ts); sin(th_n); F0_filt]; %  
S(n)
```

```
K = (1/Ts^2)*(-2*th_n+th_nm1+th_np1)*s_n'*inv(s_n*s_n')
```

```
%% Yaw Model
```

```
% This file contains a  $4 \times T$  matrix where the first row is the time stamp of  
the experiment,  
% the second row is the applied voltage, the third row is the motor's angular  
velocity,  
% and the fourth row is the pitch angle of the Aero.  
load('yaw_moi_team_5_2.mat');  
load('motor_force.mat');  
T_s = 0.1;  
t = 0:T_s:1.03980e+03;
```

```

Fs = 1/T_s;      % sampling frequency (500 Hz)
Fc = 0.1;        % cutoff frequency (Hz)
N = 2;           % filter order 2

v = data(2,:);
theta_b = data(3,:);
omega_y = data(4,:);

% Calculated Force from measured voltage
F0 = motorforce(2,:);

% Low-pass filter
[b, a] = butter(N, Fc/(Fs/2));

omega_y_filtered = filtfilt(b, a, omega_y); % Filtered angular velocity

% different data points where pitch angle was constant
plot(t, omega_y_filtered);
theta_b1 = mean(theta_b(53:814));
omega_y1 = omega_y_filtered(53:814);
F0_1 = F0(53:814);

theta_b2 = mean(theta_b(1019:2983));
omega_y2 = omega_y_filtered(1019:2983);
F0_2 = F0(1019:2983);

theta_b3 = mean(theta_b(3113:3938));
omega_y3 = omega_y_filtered(3113:3938);
F0_3 = F0(3113:3938);

theta_b4 = mean(theta_b(4001:5057));
omega_y4 = omega_y_filtered(4001:5057);
F0_4 = F0(4001:5057);

theta_b5 = mean(theta_b(5218:7157));
omega_y5 = omega_y_filtered(5218:7157);
F0_5 = F0(5218:7157);

theta_b6 = mean(theta_b(7310:8142));
omega_y6 = omega_y_filtered(7310:8142);
F0_6 = F0(7310:8142);

theta_b7 = mean(theta_b(8183:10376));
omega_y7 = omega_y_filtered(8183:10376);
F0_7 = F0(8183:10376);

% Initialize K matrix: 7 x 3
K = zeros(7, 3);

% Group Values
omega_y_all = {omega_y1, omega_y2, omega_y3, omega_y4, omega_y5, omega_y6,
omega_y7};

theta_b_all = {theta_b1, theta_b2, theta_b3, theta_b4, theta_b5, theta_b6,
theta_b7};

```

```

F0_all = {F0_1, F0_2, F0_3, F0_4, F0_5, F0_6, F0_7};

% Calculate K for all
for i = 1:7
    omega_y_n = omega_y_all{i}(2:(end-1));
    omega_y_np1 = omega_y_all{i}(3:(end));
    F0_n = F0_all{i}(2:(end-1));
    s_n = [F0_n; sign(omega_y_n); omega_y_n];

    K(i,:) = (1/T_s) * (omega_y_np1 - omega_y_n) * s_n' * inv(s_n * s_n');
end

% Convert theta_b_all cell array into numeric vector
theta_numeric = cellfun(@(x) x, theta_b_all);
theta_numeric = theta_numeric(:); % column vector

% Extract K values
K1 = K(:,1);
K2 = K(:,2);
K3 = K(:,3);

K_all = {K1, K2, K3};
names = {'K_1', 'K_2', 'K_3'};

% Loop for 2nd-order polynomial fits
for i = 1:3

    Ki = K_all{i};
    Ki = Ki(:);
    name = names{i};

    % 2nd-order polynomial fit
    p = polyfit(theta_numeric, Ki, 2);

    fprintf('%s(theta) = %.6f*theta^2 + %.6f*theta + %.6f\n', ...
            name, p(1), p(2), p(3));

    % Evaluate for plotting
    theta_fit = linspace(min(theta_numeric), max(theta_numeric), 300);
    Ki_fit = polyval(p, theta_fit);

    % Plot
    figure;
    plot(theta_numeric, Ki, 'o', 'MarkerSize', 8, 'LineWidth', 1.5);
    hold on;
    plot(theta_fit, Ki_fit, 'LineWidth', 2);

    xlabel('\theta');
    ylabel(name);
    title(['2nd-Order Fit of ', name, ' vs \theta']);
    legend('Data Points', '2nd-Order Fit', 'Location', 'Best');
    grid on;

```

end